

CBSE Class 12 Computer Science Project

Comprehensive Code Explanation Guide for function.py

Project: Student Marketplace	Target File: function.py
Database: MySQL (student_market_db)	Tech Stack: Python 3 + mysql.connector

Block 1: Module Header & Package Imports (Lines 1 – 25)

This block imports standard Python libraries and initializes the database driver.

```
import sys
import datetime

try:
    import mysql.connector
    from mysql.connector import Error
except ImportError:
    print("\n[!] 'mysql-connector-python' is not installed.")
    sys.exit(1)
```

- **import sys:** Provides system-specific parameters and the `sys.exit(1)` command to terminate execution cleanly if prerequisites are missing.
- **import datetime:** Used to fetch today's system date (`datetime.date.today()`) when saving new student listings.
- **try-except ImportError:** Gracefully verifies if `mysql.connector` is installed in the Python environment. If not found, alerts the student with installation instructions rather than crashing.

Block 2: Global Database Configuration & Category Constants (Lines 27 – 39)

Defines centralized configuration settings and validated category options.

```
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "root",
    "database": "student_market_db"
}

CATEGORIES = [
    "Books", "Calculator", "Uniform",
    "Stationery", "Sports", "Electronics", "Other"
]
```

- **DB_CONFIG (Dictionary):** Centralizes MySQL server credentials (host, username, root password, and database name). Changing credentials here updates all database functions across the whole application.
- **CATEGORIES (List):** A fixed list of standard academic categories. Used to display numbered menus and validate user inputs.

Block 3: Database Connection Handler: get_db_connection() (Lines 44 – 78)

Self-healing database connection manager with automated error recovery.

```
def get_db_connection(use_database=True):
    while True:
        try:
            if use_database:
                conn = mysql.connector.connect(
                    host=DB_CONFIG["host"], user=DB_CONFIG["user"],
                    password=DB_CONFIG["password"], database=DB_CONFIG["database"]
                )
            else:
                conn = mysql.connector.connect(
                    host=DB_CONFIG["host"], user=DB_CONFIG["user"],
                    password=DB_CONFIG["password"]
                )
            if conn.is_connected():
                return conn
        except Error as err:
            if err.errno == 1045: # Wrong password
                new_pwd = input("Enter your MySQL root password: ")
                DB_CONFIG["password"] = new_pwd
            elif err.errno == 1049: # Missing DB
                initialize_database()
                use_database = True
            elif err.errno == 2003: # MySQL server not running
                print("\n[!] ERROR: Cannot connect to MySQL server.")
                sys.exit(1)
```

- **use_database (bool):** When False, connects directly to MySQL server without selecting a database (used during initial database creation).
- **Error Handling (errno 1045):** If MySQL password is wrong, interactively prompts the user for their root password and retries.
- **Error Handling (errno 1049):** If database doesn't exist yet, automatically calls initialize_database().
- **Error Handling (errno 2003):** Warns user if MySQL service (XAMPP / MySQL Server) is stopped.

Block 4: Database & Table Initialization: `initialize_database()` (Lines 80 – 131)

Automates the entire DDL setup and pre-populates initial demo records.

```
def initialize_database():
    conn = get_db_connection(use_database=False)
    cursor = conn.cursor()
    cursor.execute("CREATE DATABASE IF NOT EXISTS student_market_db")
    cursor.execute("USE student_market_db")
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS listings (
            id INT AUTO_INCREMENT PRIMARY KEY,
            item_name VARCHAR(100) NOT NULL,
            category VARCHAR(50) NOT NULL,
            condition_status VARCHAR(30) NOT NULL,
            price DECIMAL(10,2) NOT NULL,
            seller_name VARCHAR(100) NOT NULL,
            student_id VARCHAR(50) NOT NULL,
            contact VARCHAR(50) NOT NULL,
            listed_date DATE NOT NULL,
            description TEXT,
            status VARCHAR(30) DEFAULT 'Available'
        )
    ''')
    conn.commit()
    # Auto-seed sample records if table is empty
    cursor.execute("SELECT COUNT(*) FROM listings")
    if cursor.fetchone()[0] == 0:
        cursor.executemany("INSERT INTO listings (...) VALUES (%s,...)", sample_data)
        conn.commit()
```

- **CREATE DATABASE / TABLE IF NOT EXISTS:** Guarantees the application runs out-of-the-box on any computer without manual SQL imports.
- **DECIMAL(10,2):** Accurately stores item prices with 2 decimal places.
- **AUTO_INCREMENT PRIMARY KEY:** Automatically assigns unique incremental integer IDs to every listing.
- **cursor.executemany():** Performs batch insertion of 5 starter sample listings (RD Sharma, Casio Calculator, Blazer, etc.) if the table is freshly created.

Block 5: ASCII Grid Formatter: `display_listings_table()` (Lines 133 – 155)

Formats any list of MySQL query tuples into a neat, aligned terminal table.

```
def display_listings_table(listings, title="STUDENT MARKETPLACE LISTINGS"):
    print("\n" + "="*96)
    print(f"                                {title.upper()} ({len(listings)} Items)")
    print("="*96)
    if not listings:
        print(" No items found matching criteria.")
        return
    header = f"{'ID':<6}{'Item Title':<28}{'Category':<14}{'Condition':<12}{'Price (INR)':<14}{'Seller':<12}{'Status':<10}"
    print(header)
    for item in listings:
        price_str = f"Rs. {float(item[4]):.2f}"
        print(f"#{item[0]:<5}{str(item[1])[:26]:<28}{str(item[2])[:12]:<14}..." )
```

- **Reusable UI Component:** Called by Browse, Search, Category Filter, Budget Filter, and Sort functions to ensure uniform presentation.
- **String Slicing (e.g. `[:26]`):** Truncates long item titles and descriptions so columns never wrap or distort table borders.
- **Formatted Currency:** Standardizes price display to Rs. XXX.XX.

Block 6: Browse Active Listings: `browse_all_listings()` (Lines 157 – 170)

Fetches and displays all available items in descending order of creation.

```
def browse_all_listings():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM listings WHERE status = 'Available' ORDER BY id DESC")
    rows = cursor.fetchall()
    display_listings_table(rows, "Active Items Available for Sale (MySQL)")
    cursor.close()
    conn.close()
```

- **WHERE status = 'Available':** Filters out items that have already been purchased/sold.
- **ORDER BY id DESC:** Ensures the latest listings appear at the top.
- **cursor.fetchall():** Retrieves all matching rows as a list of tuples and forwards them to `display_listings_table()`.

Block 7: Add New Listing: `add_listing()` (Lines 171 – 237)

Collects user input, validates constraints, and executes parameterized SQL INSERT.

```
def add_listing():
    item_name = input("Enter Item Title: ").strip()
    # Category selection from CATEGORIES menu
    # Condition: 1->Like New, 2->Good, 3->Fair
    price_val = float(input("Enter Expected Price: "))
    seller_name = input("Enter Seller Name: ").strip()
    student_id = input("Enter Student ID: ").strip()
    contact = input("Enter Contact Number: ").strip()
    description = input("Enter Detailed Description: ").strip()
    today_str = datetime.date.today().strftime("%Y-%m-%d")

    sql = '''INSERT INTO listings (item_name, category, condition_status,
        price, seller_name, student_id, contact, listed_date, description, status)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, 'Available')'''
    cursor.execute(sql, (item_name, category, condition, price_val, ...))
    conn.commit()
    item_id = cursor.lastrowid
```

- **Input Validation:** Checks for non-empty title and validates numeric positive price using try-except ValueError.
- **Parameterized SQL (%s):** Uses parameter tuples to completely prevent SQL injection attacks.
- **conn.commit():** Commits the transaction to make the new record permanent in MySQL.
- **cursor.lastrowid:** Retrieves the auto-generated primary key ID to confirm successful listing.

Block 8: Search & Multi-Filter Engine: `search_and_filter()` (Lines 239 – 302)

Comprehensive search engine offering 4 distinct querying modes.

```
def search_and_filter():
    choice = input("Select search type (1-4): ")
    if choice == "1": # Keyword Search
        param = f"%{kw}%"
        sql = "SELECT * FROM listings WHERE item_name LIKE %s OR description LIKE %s OR category LIKE %s"
        cursor.execute(sql, (param, param, param))
    elif choice == "2": # Category Filter
        sql = "SELECT * FROM listings WHERE category = %s"
        cursor.execute(sql, (target_cat,))
    elif choice == "3": # Budget Range Filter
        sql = "SELECT * FROM listings WHERE price <= %s AND status = 'Available' ORDER BY price ASC"
        cursor.execute(sql, (max_p,))
    elif choice == "4": # All Archive
        cursor.execute("SELECT * FROM listings ORDER BY id DESC")
```

- **Mode 1 (Keyword Search):** Uses SQL wildcard pattern matching (LIKE %kw%) across title, category, and description.
- **Mode 2 (Category Filter):** Performs exact match lookup based on user selection.
- **Mode 3 (Budget Constraint):** Filters active items by maximum budget (price <= max_p) sorted cheapest first.
- **Mode 4 (Complete Archives):** Displays all historical listings (both Available and Sold).

Block 9: Inspect Single Listing: view_listing_details() (Lines 298 – 335)

Retrieves and displays full details including seller contact information for a specific item.

```
def view_listing_details():
    item_id = input("Enter Listing ID to view: ").strip()
    cursor.execute("SELECT * FROM listings WHERE id = %s", (item_id,))
    target = cursor.fetchone()
    # Prints formatted card with seller contact and description
```

- **cursor.fetchone():** Fetches the single matching row tuple for the requested ID.
- **Card View Display:** Shows full description, condition, price, seller name, student roll number, and contact info so prospective buyers can connect.

Block 10: Dynamic Sorting Engine: sort_listings() (Lines 333 – 363)

Delegates sorting directly to the MySQL database engine for high efficiency.

```
def sort_listings():
    sort_choice = input("Enter sorting option (1-3): ")
    if sort_choice == "1":
        cursor.execute("SELECT * FROM listings WHERE status = 'Available' ORDER BY price ASC")
    elif sort_choice == "2":
        cursor.execute("SELECT * FROM listings WHERE status = 'Available' ORDER BY price DESC")
    )
    elif sort_choice == "3":
        cursor.execute("SELECT * FROM listings WHERE status = 'Available' ORDER BY listed_date DESC, id DESC")
```

- **ORDER BY price ASC:** Sorts from lowest to highest price.
- **ORDER BY price DESC:** Sorts from highest to lowest price.
- **ORDER BY listed_date DESC, id DESC:** Sorts most recently listed items first.

Block 11: Transaction Completion: mark_item_sold() (Lines 365 – 408)

Manages item availability lifecycle and status transitions.

```
def mark_item_sold():
    item_id = input("Enter Listing ID: ").strip()
    cursor.execute("SELECT item_name, status FROM listings WHERE id = %s", (item_id,))
    target = cursor.fetchone()
    if target[1] == "Sold":
        # Option to reopen listing to Available
        cursor.execute("UPDATE listings SET status = 'Available' WHERE id = %s", (item_id,))
    else:
        # Confirm and mark Sold
        cursor.execute("UPDATE listings SET status = 'Sold' WHERE id = %s", (item_id,))
    conn.commit()
```

- **SQL UPDATE Operation:** Updates status = 'Sold' upon successful transaction.
- **Reopening Capability:** If an item was marked sold accidentally, allows toggling status back to Available.
- **Safety Verification:** Prompts confirmation before modifying data.

Block 12: Record Removal: delete_listing() (Lines 410 – 443)

Executes permanent deletion of a listing from the MySQL database.

```
def delete_listing():
    item_id = input("Enter Listing ID to delete: ").strip()
    cursor.execute("SELECT item_name FROM listings WHERE id = %s", (item_id,))
    target = cursor.fetchone()
    if target:
        confirm = input(f"Permanently remove #{item_id}? (y/n): ")
        if confirm == 'y':
            cursor.execute("DELETE FROM listings WHERE id = %s", (item_id,))
            conn.commit()
```

- **Existence Check:** Verifies the record exists before attempting deletion.
- **Explicit Confirmation:** Prevents accidental permanent data loss.
- **SQL DELETE Operation:** Executes parameterized DELETE FROM listings WHERE id = %s.

Block 13: Statistical Analytics & Insights: marketplace_insights() (Lines 440 – 488)

Leverages MySQL aggregate functions to generate comprehensive business metrics.

```
def marketplace_insights():
    cursor.execute("SELECT COUNT(*) FROM listings")
    cursor.execute("SELECT COUNT(*) FROM listings WHERE status = 'Available'")
    cursor.execute("SELECT COUNT(*) FROM listings WHERE status = 'Sold'")
    cursor.execute("SELECT AVG(price) FROM listings WHERE status = 'Available'")
    cursor.execute("SELECT id, item_name, price FROM listings ... ORDER BY price ASC LIMIT 1")
    cursor.execute("SELECT id, item_name, price FROM listings ... ORDER BY price DESC LIMIT 1")
)
    cursor.execute("SELECT category, COUNT(*) FROM listings WHERE status = 'Available' GROUP BY category ORDER BY COUNT(*) DESC")
```

- **SQL COUNT(*):** Computes total, available, and sold item quantities.
- **SQL AVG(price):** Computes mathematical average listing price across active inventory.
- **Price Bounds (MIN/MAX via LIMIT 1):** Highlights the cheapest and most expensive item in the market.
- **SQL GROUP BY category:** Produces a category-wise inventory breakdown sorted by popularity.

Class 12 CS Viva Voce: Key Defense Points

- ✓ **Modular Architecture:** Code is separated into main.py (presentation/menu) and function.py (business and database logic).
- ✓ **Driver Used:** mysql.connector handles socket connections and SQL query transmission.
- ✓ **Auto-Initialization:** Uses CREATE DATABASE IF NOT EXISTS and CREATE TABLE IF NOT EXISTS so examiners don't need to manually import SQL files.
- ✓ **Security:** All SQL queries use parameterized %s placeholders to completely prevent SQL Injection.
- ✓ **Transaction Persistence:** Every INSERT, UPDATE, and DELETE is followed by conn.commit() to ensure changes are written to disk.
- ✓ **SQL Concepts Demonstrated:** DDL (CREATE), DML (INSERT, UPDATE, DELETE), DQL (SELECT with WHERE, LIKE, ORDER BY, GROUP BY), and Aggregate functions (COUNT, AVG).